

ALGOTHON'26

PROBLEM STATEMENTS

12 Problem Statements • 6 Domains • Unique PS IDs

DOMAINS

- 1. AI / ML
- 2. Web Development
- 3. Blockchain
- 4. Automation
- 5. Data Science
- 6. Cybersecurity

=====

1. AI / ML

AI Resume & Job Matching System

PS ID: ALG-AI-01

=====

Problem Overview:

Recruiters receive hundreds of resumes for a single role. Build a system that analyzes resumes against a job description, ranks candidates, and explains why each candidate matches.

What to Build:

Build a web platform where recruiters upload multiple resumes and a job description. Extract skills, education, and experience; score candidates; search, filter, and rank results; and explain the match.

Must Have:

- * Resume upload
- * Job-description input
- * Information extraction
- * Candidate scoring/ranking
- * Search and filters
- * Explainable match result

Innovation / Bonus:

Detect unsupported or contradictory claims instead of blindly rewarding keyword matches.

Suggested Tech:

Python, NLP, LLMs, embeddings/vector search, OCR, database, React/Next.js

Judging Focus:

Accuracy, explainability, useful ranking, and robustness to messy resumes.

=====

2. AI / ML

Intelligent Document Investigator

PS ID: ALG-AI-02

=====

Problem Overview:

Information is often scattered across PDFs, images, and text documents. Users need answers without manually reading every document.

What to Build:

Build a document investigation platform that accepts multiple documents and answers natural-language questions with supporting sources.

Must Have:

- * Multiple document formats
- * Extraction/indexing
- * Natural-language Q&A
- * Source/section references
- * Conflict detection
- * Uncertainty handling

Innovation / Bonus:

Identify conflicting documents and communicate uncertainty instead of confidently returning one unsupported answer.

Suggested Tech:

RAG, NLP, OCR, embeddings, vector DB, Python, web framework

Judging Focus:

Answer quality, source grounding, conflict handling, and usable investigation workflow.

=====

3. WEB DEVELOPMENT

Collaborative Project Workspace

PS ID: ALG-WEB-01

=====

Problem Overview:

Remote teams often use disconnected tools for tasks, comments, files, and progress.
Build one collaborative workspace.

What to Build:

Build a real-time project management workspace where teams create projects, assign tasks, comment, attach files, and track progress.

Must Have:

- * Projects and tasks
- * Task assignment/status
- * Comments/files
- * Deadlines/progress
- * Dashboard
- * Real-time updates

Innovation / Bonus:

Simultaneous edits to the same task must not silently lose updates.

Suggested Tech:

React/Next.js, WebSockets, Node/FastAPI, PostgreSQL/Supabase

Judging Focus:

Real-time reliability, collaboration UX, data consistency, and completeness.

=====

4. WEB DEVELOPMENT

Offline-First Application

PS ID: ALG-WEB-02

=====

Problem Overview:

Many web applications become unusable when connectivity is unstable. Build an app that remains useful offline and synchronizes safely later.

What to Build:

Build an offline-first web application that caches data, supports create/edit/delete operations offline, queues changes, and synchronizes when connectivity returns.

Must Have:

* Offline cache

- * Offline CRUD
- * Pending changes
- * Connectivity detection
- * Synchronization
- * Sync status

Innovation / Bonus:

Conflicting edits from two offline devices must be detected and handled without silent overwrite.

Suggested Tech:

PWA, service workers, IndexedDB/local storage, React/Next.js, backend APIs

Judging Focus:

Offline behavior, synchronization correctness, conflict handling, and UX.

=====

5. BLOCKCHAIN

Digital Certificate Verification

PS ID: ALG-BC-01

=====

Problem Overview:

Fake or modified certificates are difficult to verify. Build a blockchain-backed certificate issuing and verification system.

What to Build:

Build a system where an issuer creates a certificate record, gives it a unique verification ID/QR link, and anyone can verify its status and authenticity.

Must Have:

- * Issuer workflow
- * Unique verification ID
- * QR/link verification
- * Public verification page
- * Status/revocation
- * Integrity checking

Innovation / Bonus:

Verify that certificate data has not changed and that it originated from the claimed issuer.

Suggested Tech:

Solidity/EVM or equivalent blockchain, hashing, smart contracts, wallet integration, web3 libraries

Judging Focus:

Actual on-chain verification, integrity, revocation logic, and usability.

=====

6. BLOCKCHAIN

Blockchain Escrow System

PS ID: ALG-BC-02

=====

Problem Overview:

Unknown buyers and sellers need a trusted way to transact without relying on a centralized intermediary. Build a blockchain escrow platform.

What to Build:

Build a decentralized escrow platform where a smart contract locks funds and releases, refunds, or holds them according to predefined conditions.

Must Have:

- * Wallet connection
- * Create escrow
- * Deposit/lock funds
- * Seller delivery status
- * Buyer release
- * Deadline refund
- * Basic dispute flow
- * Transaction history

Innovation / Bonus:

Milestone payments, multi-party escrow, reputation, or stronger dispute handling.

Suggested Tech:

Solidity, Ethereum-compatible testnet, Hardhat/Foundry, MetaMask, ethers.js/viem/wagmi, React/Next.js

Judging Focus:

Smart contract security, state transitions, real end-to-end fund flow, and edge cases.

=====

7. AUTOMATION

Visual Workflow Automation

PS ID: ALG-AUTO-01

=====

Problem Overview:

People repeat sequences of digital tasks across tools. Build a visual automation platform that lets users define and execute workflows.

What to Build:

Build a visual workflow builder where users drag nodes, connect triggers/actions/conditions, and execute reusable workflows.

Must Have:

- * Visual workflow editor
- * Triggers
- * Actions
- * Conditions
- * Data transformation

- * Actual execution
- * Execution status/logs
- * Save/load workflows

Innovation / Bonus:

Reusable workflows, retries, scheduling, webhooks, parallel branches, or natural-language workflow capabilities.

Suggested Tech:

React Flow or similar, Node/Python, queues, database, REST APIs

Judging Focus:

Real execution engine, correct data passing, reliability, and workflow UX.

=====

8. AUTOMATION

Smart Application Processing

PS ID: ALG-AUTO-02

=====

Problem Overview:

Organizations manually process large numbers of applications and documents. Build an automated processing pipeline.

What to Build:

Build a system that accepts applications, validates information, processes documents, categorizes submissions, stores results, and communicates outcomes.

Must Have:

- * Application form
- * Validation
- * Missing/inconsistent field detection
- * Document processing
- * Categorization
- * Storage
- * Notifications

Innovation / Bonus:

Incomplete information should be explained and routed for correction instead of simply being rejected.

Suggested Tech:

Workflow automation, document processing, APIs, database, email/notification, optional AI

Judging Focus:

Automation completeness, exception handling, accuracy, and communication.

=====

9. DATA SCIENCE

The Mystery Dataset

PS ID: ALG-DATA-01

=====

Problem Overview:

Teams receive a large dataset without a predefined business question. They must investigate it and discover meaningful patterns.

What to Build:

Clean and explore the dataset, identify relationships and anomalies, formulate hypotheses, visualize findings, and produce analytical or predictive results.

Must Have:

- * Data cleaning
- * Variable understanding
- * Relationship analysis
- * Anomaly detection
- * Visualizations
- * Hypothesis
- * Analytical/predictive result

Innovation / Bonus:

The strongest submission should explain a meaningful discovery that was not obvious at first glance.

Suggested Tech:

Python, pandas, NumPy, scikit-learn, visualization libraries

Judging Focus:

Quality of analysis, insight depth, visualization, and evidence-backed conclusions.

=====

10. DATA SCIENCE

Predict What Happens Next

PS ID: ALG-DATA-02

=====

Problem Overview:

Historical data can reveal patterns that help anticipate future behavior. Build a predictive pipeline from historical data.

What to Build:

Explore and clean data, engineer features, select and train models, validate them with suitable metrics, and generate predictions for unseen data.

Must Have:

- * Exploration/cleaning
- * Feature engineering
- * Model selection
- * Training/validation
- * Metrics
- * Unseen predictions

Innovation / Bonus:

Use a hidden final evaluation dataset so teams cannot manually hardcode or prepare answers for the test cases.

Suggested Tech:

Python, pandas, scikit-learn, XGBoost/LightGBM, visualization

Judging Focus:

Model quality, validation methodology, feature reasoning, and robustness.

=====

11. CYBERSECURITY

Find the Intruder

PS ID: ALG-CYBER-01

=====

Problem Overview:

Thousands of security events can hide an attacker among normal activity. Build a system that identifies suspicious behavior from logs.

What to Build:

Analyze authentication, network, or server logs to identify suspicious users/IPs, connect related events, and create an incident timeline.

Must Have:

- * Log ingestion
- * Anomaly detection/rules
- * Suspicious user/IP detection
- * Event grouping
- * Incident timeline
- * Evidence

Innovation / Bonus:

Connect multiple events and explain the likely attack sequence instead of flagging isolated events only.

Suggested Tech:

Python, log analysis, anomaly detection, rules/ML, dashboard, database/search

Judging Focus:

Detection quality, evidence, correlation across events, and explainability.

=====

12. CYBERSECURITY

Secure the Application

PS ID: ALG-CYBER-02

=====

Problem Overview:

Deliberately vulnerable web applications contain weaknesses that attackers can exploit. Identify, demonstrate safely, fix, and verify them.

What to Build:

Inspect the application, identify security weaknesses, safely demonstrate the issue, implement fixes, and retest legitimate functionality.

Must Have:

- * Authentication/input inspection
- * Vulnerability identification
- * Safe demonstration
- * Secure fixes
- * Retesting
- * Root-cause documentation

Innovation / Bonus:

Security fixes must not break legitimate application functionality. Teams should explain the root cause, not just patch symptoms.

Suggested Tech:

Web security, secure coding, OWASP principles, backend development, testing

Judging Focus:

Correct vulnerability identification, safe testing, quality of fixes, and regression testing.

=====

COMMON SUBMISSION EXPECTATIONS

=====

Applies to all 12 problem statements:

- * Working project with a deployed demo where practical.
- * Source-code repository with a clear README.
- * Architecture diagram and explanation of major technical decisions.
- * Demonstration of the core workflow required by the selected PS.
- * Testing evidence and handling of important edge cases.
- * Known limitations and future improvements.
- * Disclosure of external APIs, datasets, and AI-assisted components used.
- * Teams may use equivalent technologies unless a PS specifically requires a particular technology.

=====

EVALUATION FRAMEWORK

=====

Common judging criteria:

1. Functionality & Completion — 30%

Does the core solution actually work?

2. Technical Implementation — 20%

Quality, architecture, and appropriate technology.

3. Innovation & Problem Understanding — 20%

Originality and depth of solution.

4. User Experience / Presentation — 15%

Clarity, usability, and demo quality.

5. Testing, Edge Cases & Reliability — 15%

Robustness beyond the happy path.

=====

ALGOTHON'26 PRINCIPLE

=====

AI MAY HELP YOU BUILD IT.
BUT YOU STILL HAVE TO MAKE IT WORK.

12 PS • 6 DOMAINS • 1 HACKATHON